AI Pioneers

CCA-F | Module 4

Lab 4.2

Enforcing Structure: tool_use Schemas with Validation & Retry

Scenario: A Recruiting Candidate-Screening Evaluator (strong_hire / hire / no_hire)

Module	M4 — Prompt Engineering & Structured Output
Lab type	Hands-on · Self-paced or Instructor-Led · Claude API (Python)
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.9+ · anthropic SDK · `ANTHROPIC_API_KEY`
Sections	S3 (Structured Output using `tool_use` & JSON Schema) S4 (Validation, Retry & Feedback Loops)

Lab Objectives

By the end of this lab you will be able to:

- **Force structured output through tool_use and JSON Schema** — guarantee every screening evaluation returns a valid `{name, recommendation, score, reason}` object so recruiters can sort and act on it without parsing prose.
- **Add schema plus semantic validation to catch bad data** — reject a score outside 0–10 with the schema, and enforce policy the schema cannot express (e.g. `strong_hire ⇒ score ≥ 8`) in your own validator.
- **Build retry-and-feedback loops that self-correct** — feed the validation error back as a `tool_result` with `is_error=True` so the model fixes the evaluation on the next turn, capped so the loop always terminates.

1. Overview

A free-text candidate write-up is unsortable. A recruiting screen needs a predictable object every time so a downstream UI can filter, rank, and act. This lab makes the output reliably structured against a candidate-screening task that produces, for each application, a record of {name, recommendation, score, reason}. First you force the shape with a tool_use schema and tool_choice so the tool's arguments are your structured object — no parsing prose, no stray preambles. Next you add a validator for the rules the JSON Schema cannot express (cross-field policy like strong_hire ⇒

score ≥ 8). Finally you wrap it in a retry-and-feedback loop that returns the validation error to the model as a tool_result with is_error=True until the evaluation is valid or you hit the retry cap. Each exercise is a self-contained Python script you run against the Claude API. This lab covers Module 4 sections S3 and S4.

SN	Section	What you will do	Core idea
Ex 1	**S3 — tool_use + JSON Schema**	Complete a tool's `input_schema` and force `tool_choice` so every response is the structured object.	Make the schema the contract; the tool's arguments ARE the structured output.
Ex 2	**S4 — Schema + semantic validation**	Implement `validate()` with type/enum/range checks plus cross-field policy.	Schema enforces shape; your code enforces meaning.
Ex 3	**S4 — Retry-and-feedback loop**	On invalid output, return the error as a `tool_result(is_error=True)` and let the model self-correct, capped.	Show the model what was wrong; bound the loop so it always terminates.

2. Scenario

You are building the screening step for a recruiting platform. Each application becomes a structured evaluation that feeds a recruiter dashboard, so the output must be machine-readable — every record has the same four fields with the same types — and must be consistent — a strong_hire scored a 5 cannot slip through. A free-text rationale on its own is fine for a human; it is useless for sorting, filtering, or alerting.

Your job is to make the evaluator produce a guaranteed valid object every time, with prompting and tool schemas — no fine-tuning. You will define the shape as a tool, validate the cross-field policy the schema cannot express, and add a retry loop that hands the model the specific error so it can correct itself.

You will:

- Define an `EVALUATE_TOOL` whose `input_schema` describes the target record (`name`, `recommendation` enum, `score` 0–10, `reason`), force `tool_choice` to the named tool, and read the structured object straight off `block.input`.
- Implement a `validate(payload)` function that returns `(ok, errors)` and catches both type errors and the cross-field policy `strong_hire ⇒ score ≥ 8` / `no_hire ⇒ score ≤ 4`.
- Wrap the call in a loop that, on validation failure, appends the assistant turn and a `tool_result` with `is_error=True` carrying the error string, then retries up to a cap.

The target record

```
{ "name": str, "recommendation": "strong_hire" | "hire" | "no_hire", "score": int 0-10, "reason": str }
```

Cross-field policy (not expressible in JSON Schema): `strong_hire` must score `>= 8`; `no_hire` must score `<= 4`.

Why these three together?

They are three layers of reliability. The tool schema fixes the shape — every response is a typed object. The validator fixes the meaning — cross-field rules the schema cannot express are checked in code. The retry loop fixes the failure mode — when the model produces an inconsistent evaluation, it sees the specific error and corrects it. Schema without semantic validation lets policy-violating records through. Validation without a feedback loop just rejects them. Together they make the output structured, correct, and self-healing within a bounded retry budget.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Building with the Claude API	Tool schemas · Tool functions · Structured data	**S3**
Building with the Claude API	Code based grading · Sending tool results	**S4**

Module 4 uses the Building with the Claude API course on Skilljar. This lab is mandatory; complete the lessons above before the session. The instructor will not re-teach the basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open https://www.bluelabs.studio/, locate your VM, click Start, then Connect.
- Unzip the lab bundle and enter it:

```
unzip lab-4.2-structured-output.zip
cd lab-4.2-structured-output
```

- Create a Python environment and install the Anthropic SDK:

```
python -m venv .venv && source .venv/bin/activate   # Windows:
.venv\Scripts\activate
pip install -r requirements.txt                     # anthropic>=0.40.0
```

- Add your API key and load it into the environment:

```
cp .env.example .env          # paste your key into .env
export $(grep -v '^#' .env | xargs)
```

- Confirm Exercise 2's offline check runs without an API key. On the fresh starter it reports "validate() is not implemented yet" — that is expected; it still confirms the offline path runs, and it will pass once you complete Exercise 2:

```
python exercise_2_validation.py --check
```

- Confirm Exercise 1 runs live — this makes a real API call, so it verifies your key and SDK. Once you complete its input_schema, each candidate prints as a clean four-field payload:

```
python exercise_1_tool_schema.py
```

Model, cost & safety

The scripts read the model from `ANTHROPIC_MODEL` (default `claude-sonnet-4-6`) and the key from `ANTHROPIC_API_KEY`. Each script makes a handful of small API calls, so cost is minimal. Treat the candidate descriptions as professional screening notes — short, paraphrased role summaries. Keep `max_tokens` tight (around 300) so the tool call returns quickly and the structured payload is easy to inspect.

4. Exercises

Each exercise is one self-contained file you run. Open the starter, fill in the TODO, run it. Exercises 2 and 3 each ship a no-API offline mode (`--check` / `--demo`) so you can verify your logic before spending a single API call.

Exercise 1: Force structured output with tool_use + JSON Schema (S3) ~15 min

Background

Asking the model to "return JSON" in prose is unreliable — you get markdown fences, stray preambles, missing fields, and the occasional invented one. Defining the target as a tool's `input_schema` and forcing `tool_choice = {"type": "tool", "name": "record_evaluation"}` flips the contract: the model's tool arguments ARE your structured object. The shape is guaranteed because the API rejects calls that don't match the schema.

Task

Open `exercise_1_tool_schema.py` and complete `EVALUATE_TOOL`'s `input_schema` so it describes the target record exactly, then run it on three candidates and observe a clean structured payload each time.

Step 1 — Inspect the target record

Every evaluation must contain exactly these four fields, with the listed types and constraints. Mark all four required.

Field	Type	Constraint
`name`	string	Candidate's name; non-empty.
`recommendation`	string (enum)	one of `strong_hire`, `hire`, `no_hire`
`score`	integer	`minimum: 0`, `maximum: 10`
`reason`	string	One-sentence justification tied to the role.

Step 2 — Complete the input_schema

Fill in the TODO in the starter. The target solution looks like this:

```
EVALUATE_TOOL = {
    "name": "record_evaluation",
    "description": "Record a structured screening evaluation for one job
candidate.",
    "input_schema": {
        "type": "object",
        "properties": {
            "name": {"type": "string", "description": "Candidate name."},
            "recommendation": {
                "type": "string",
                "enum": ["strong_hire", "hire", "no_hire"],
                "description": "Screening recommendation.",
            },
            "score": {
                "type": "integer", "minimum": 0, "maximum": 10,
                "description": "Overall fit, 0 (poor) to 10 (excellent).",
            },
            "reason": {"type": "string", "description": "One-sentence
justification."},
        },
        "required": ["name", "recommendation", "score", "reason"],
    },
}
```

Step 3 — Force the tool and read the structured object

The call sets `tool_choice={"type": "tool", "name": "record_evaluation"}` so the model must call the tool. The structured payload is on the `tool_use` block's `.input`:

```
msg = client.messages.create(
    model=MODEL, max_tokens=300, tools=[EVALUATE_TOOL],
    tool_choice={"type": "tool", "name": "record_evaluation"},
    messages=[{"role": "user", "content": f"Evaluate this
candidate:\n{candidate}"}],
)
for block in msg.content:
    if block.type == "tool_use":
        return block.input   # this IS the structured object
```

Step 4 — Run and inspect

```
python exercise_1_tool_schema.py
```

Expected: three lines of JSON, one per candidate, each with all four fields populated and the score inside 0–10. No prose, no markdown fences, no missing keys.

> **What good looks like**
>
> Every payload is a dict with exactly `name`, `recommendation` (one of the three enum values), `score` (an integer 0–10), and a non-empty `reason`. You never write a regex or `json.loads`; the tool's arguments are already the object you wanted.

Reflection Questions

- Why is a tool's `input_schema` plus `tool_choice` more reliable for structured output than asking the model to "reply in JSON"?
- The enum `["strong_hire", "hire", "no_hire"]` lives in the schema, while `strong_hire ⇒ score ≥ 8` does not. What kinds of rules belong in a JSON Schema, and what kinds don't?
- If you removed `"required": ["name", "recommendation", "score", "reason"]`, how would downstream code break, and what would still work?

Exercise 2: Schema plus semantic validation (S4) ~15 min

Background

A JSON Schema is a syntax check: it can say "score is an integer 0–10" and "recommendation is one of three values," but it cannot say "a `strong_hire` must score at least 8." Cross-field policy lives in code. Your validator is the second gate, after the schema, and the only one that can block a structurally valid but semantically inconsistent record.

Task

Open `exercise_2_validation.py` and implement `validate(payload)` so it returns `(ok: bool, errors: list[str])`. It must check basic shape (defensively, because a future caller might hand it anything) and the two cross-field rules. Test it offline first with `--check`, then run it live.

Step 1 — The rules to enforce

Rule	Error message (suggestion)
`name` is a non-empty string	`"name must be a non-empty string"`
`recommendation` ∈ `{strong_hire, hire, no_hire}`	`"recommendation must be one of [...]"`

Rule	Error message (suggestion)
`score` is an integer in 0..10	`"score must be an integer"` / `"score must be between 0 and 10"`
`reason` is a non-empty string	`"reason must be a non-empty string"`
`strong_hire` ⇒ `score ≥ 8`	`"a 'strong_hire' must score >= 8"`
`no_hire` ⇒ `score ≤ 4`	`"a 'no_hire' must score <= 4"`

Step 2 — Implement validate()

Return the tuple — don't raise. Watch the `bool` vs `int` trap: in Python, `isinstance(True, int)` is `True`, so accept `score=True` silently if you're not careful. Filter `bool` out explicitly. The target solution:

```
def validate(payload):
    """Return (ok: bool, errors: list[str])."""
    errors = []
    if not isinstance(payload, dict):
        return False, ["payload is not an object"]
    name = payload.get("name")
    if not isinstance(name, str) or not name.strip():
        errors.append("name must be a non-empty string")
    rec = payload.get("recommendation")
    if rec not in RECS:
        errors.append(f"recommendation must be one of {sorted(RECS)}")
    score = payload.get("score")
    is_int = isinstance(score, int) and not isinstance(score, bool)
    if not is_int:
        errors.append("score must be an integer")
    elif not (0 <= score <= 10):
        errors.append("score must be between 0 and 10")
    reason = payload.get("reason")
    if not isinstance(reason, str) or not reason.strip():
        errors.append("reason must be a non-empty string")
    if rec == "strong_hire" and is_int and score < 8:
        errors.append("a 'strong_hire' must score >= 8")
    if rec == "no_hire" and is_int and score > 4:
        errors.append("a 'no_hire' must score <= 4")
    return (len(errors) == 0), errors
```

Step 3 — Offline check, then live run

```
python exercise_2_validation.py --check    # no API key needed
python exercise_2_validation.py            # live, runs validate() on real model
output
```

The `--check` path runs three fixtures: a good record (passes), a structurally bad record (empty name, invalid enum, score out of range), and a structurally valid but policy-violating record (a `strong_hire` scored 4). All three assertions must pass before you spend an API call.

What good looks like

Offline: the `check()` assertions pass and the printed errors name the right rules. Live: each model evaluation is structurally well-formed (the schema guaranteed that) and your validator either reports `valid: True` or flags the specific rule that failed.

Reflection Questions

- Why is `strong_hire ⇒ score ≥ 8` enforced in `validate()` instead of in the JSON Schema?
- The validator returns `(ok, errors)` instead of raising. Why is a tuple a better contract here than an exception?
- Why test `not isinstance(score, bool)` even though we already test `isinstance(score, int)`?

Exercise 3: Retry-and-feedback loop that self-corrects (S4) ~15 min

Background

When the validator rejects a payload, don't just drop it — tell the model exactly what was wrong and let it try again. The Anthropic API has a built-in channel for this: append the assistant's previous turn, then append a user turn whose content is a `tool_result` with `is_error=True` and the error string. The model sees its own tool call followed by the failure, and corrects on the next attempt. Cap the retries so a stubborn case can't run forever.

Task

Complete the body of `assess_with_retry` in `exercise_3_retry_loop.py` so it loops up to `max_attempts` times, validating each tool call and, on failure, feeding the error back as a `tool_result`. Step through the loop offline with `--demo` first.

Step 1 — The loop, in pseudocode

```
messages = [user_message]
for attempt in 1..max_attempts:
    resp = client.messages.create(... tools=[EVALUATE_TOOL], tool_choice=...,
messages=messages)
    tool_use = first tool_use block in resp.content
    ok, errors = validate(tool_use.input)
    if ok: return tool_use.input, []
    # feed the failure back
    messages.append({role: assistant, content: resp.content})
    messages.append({role: user, content: [{
        type: tool_result, tool_use_id: tool_use.id, is_error: True,
        content: "Validation failed: ...; Call record_evaluation again with
corrected values."
    }]})
return last_payload, errors   # gave up after the cap
```

Step 2 — The exact tool_result shape

The `tool_use_id` must echo the id of the assistant's `tool_use` block (otherwise the API rejects the turn). The `content` is a string with the validator's joined error message, ending with an instruction to call the tool again:

```
{
  "type": "tool_result",
  "tool_use_id": tool_use.id,
  "is_error": True,
  "content": (
    "Validation failed: " + "; ".join(errors) +
    ". Call record_evaluation again with corrected values."
  ),
}
```

Step 3 — Watch the loop offline

```
python exercise_3_retry_loop.py --demo
```

The `--demo` path simulates two model attempts on the candidate "Alex Park": attempt 1 is a `strong_hire` scored 5 (policy violation), so the loop builds the exact error string and prints what would be fed back. Attempt 2 returns a corrected payload (score 9) and the loop stops. No API calls; the goal is to see the feedback mechanism end-to-end.

Step 4 — Run live

```
python exercise_3_retry_loop.py
```

Expected output: a strong candidate (Alex Park, 10 years, led re-architecture) typically passes on attempt 1 — printed as `attempt 1: valid` — and the final line shows the structured payload with empty errors. If the model produces an inconsistent first attempt, you'll see it flagged and corrected on attempt 2.

What good looks like

The loop prints `attempt N: valid` within the retry budget; the final payload satisfies both the schema and your validator. On a deliberately tricky case, attempt 1 is flagged (`attempt 1: invalid -> ["a 'strong_hire' must score >= 8"]`), the model receives the error via `tool_result(is_error=True)`, and attempt 2 returns a corrected payload. The `max_attempts` cap guarantees termination even if the model never gets it right.

Reflection Questions

- Why is the validation error returned as a `tool_result` with `is_error=True` rather than as a normal user message describing the problem?
- Why do you need to append the assistant's previous `resp.content` to `messages` before adding the `tool_result`?
- Why cap the retries instead of looping until the output is valid? What is the failure mode of an uncapped loop?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	How does a tool `input_schema` combined with `tool_choice` force structured output, and why is reading `block.input` safer than parsing prose?
2	Which kinds of rules belong in a JSON Schema (syntax) versus in your validator (semantics / cross-field policy)?
3	Why does `validate()` return `(ok, errors)` instead of raising, and how does that shape the retry loop?
4	What goes into the `tool_result` turn that lets the model self-correct, and why `is_error=True`?
5	Why cap `max_attempts`, and what should the function return when the cap is hit?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Asking for JSON in prose.	Markdown fences, preambles, and missing fields creep in. Define the shape as a tool's `input_schema` and force `tool_choice` to the named tool — the tool arguments ARE the structured object.
Putting cross-field policy in the schema.	Basic JSON Schema can't express `strong_hire ⇒ score ≥ 8`. Keep types/enum/range in the schema and policy rules in `validate()`.
Raising from the validator.	Exceptions make the retry loop awkward. Return `(ok, errors)` so the caller can feed the message back to the model and keep going.
Forgetting that `bool` is an `int`.	`isinstance(True, int)` is `True`. If you don't filter `bool` out, `score=True` passes silently as 1. Test `isinstance(score, int) and not isinstance(score, bool)`.
Sending the error as a plain user message.	The model loses the link to its previous tool call. Use a `tool_result` with the matching `tool_use_id` and `is_error=True` so the failure is attached to the right turn.
Uncapped retry loop.	A truly stuck case can spend tokens forever. Cap with `max_attempts` and return the last payload plus errors so callers can decide whether to escalate.

6. Quick Reference

6.1 Tool Schema + Forced tool_choice

```
# Define the target object AS a tool's input_schema, then force the tool.
EVALUATE_TOOL = {
  "name": "record_evaluation",
  "description": "Record a structured screening evaluation for one job candidate.",
  "input_schema": {
    "type": "object",
    "properties": {
      "name":           {"type": "string"},
      "recommendation": {"type": "string", "enum":
["strong_hire","hire","no_hire"]},
      "score":          {"type": "integer", "minimum": 0, "maximum": 10},
      "reason":         {"type": "string"},
    },
    "required": ["name","recommendation","score","reason"],
  },
}
msg = client.messages.create(
    model=MODEL, max_tokens=300, tools=[EVALUATE_TOOL],
    tool_choice={"type": "tool", "name": "record_evaluation"},
    messages=[...],
)
payload = next(b.input for b in msg.content if b.type == "tool_use")
```

6.2 Schema vs. Semantic Validation

```
# Schema (in input_schema) handles SYNTAX:
#   types, enum values, integer minimum/maximum, required fields.
# validate() handles SEMANTICS (cross-field, policy):
#   strong_hire => score >= 8
#   no_hire     => score <= 4
if rec == "strong_hire" and is_int and score < 8:
    errors.append("a 'strong_hire' must score >= 8")
if rec == "no_hire" and is_int and score > 4:
    errors.append("a 'no_hire' must score <= 4")
```

6.3 Validator Contract: return (ok, errors)

```
def validate(payload) -> tuple[bool, list[str]]:
    """Defensive: never raise; accept anything, return a structured verdict."""
    errors = []
    if not isinstance(payload, dict): return False, ["payload is not an object"]
    # ... per-field checks ...
    # Filter bool out of int checks: isinstance(True, int) is True!
    is_int = isinstance(score, int) and not isinstance(score, bool)
    return (len(errors) == 0), errors
```

6.4 Retry Loop with tool_result Feedback

```
messages = [{"role": "user", "content": user_prompt}]
for attempt in range(1, max_attempts + 1):
    resp = client.messages.create(model=MODEL, tools=[EVALUATE_TOOL],

tool_choice={"type":"tool","name":"record_evaluation"},
                                  messages=messages, max_tokens=300)
    tool_use = next((b for b in resp.content if b.type == "tool_use"), None)
    ok, errors = validate(tool_use.input)
    if ok: return tool_use.input, []
    messages.append({"role": "assistant", "content": resp.content})
    messages.append({"role": "user", "content": [{
        "type": "tool_result", "tool_use_id": tool_use.id, "is_error": True,
        "content": "Validation failed: " + "; ".join(errors) +
                   ". Call record_evaluation again with corrected values.",
    }]})
return last_payload, errors   # cap reached
```

6.5 File Inventory

File	Section	Purpose
`exercise_1_tool_schema.py`	**S3**	Define `EVALUATE_TOOL.input_schema`; force `tool_choice`; read `block.input`.
`exercise_2_validation.py`	**S4**	Implement `validate()`; runs `--check` offline and live.
`exercise_3_retry_loop.py`	**S4**	Wrap `validate()` in a retry loop that feeds the error back via `tool_result(is_error=True)`; runs `--demo` offline.
`.env.example`, `requirements.txt`	**setup**	API key template and the `anthropic` SDK dependency.

6.6 Further Reading

- Tool use overview: https://docs.claude.com/en/docs/build-with-claude/tool-use
- How to implement tool use: https://docs.claude.com/en/docs/build-with-claude/tool-use/implement-tool-use
- Skilljar — Building with the Claude API: Tool schemas; Tool functions; Structured data (S3)
- Skilljar — Building with the Claude API: Code based grading; Sending tool results (S4)